

Guide Administrateur

Thesium Desk

Version 1.0 — Mars 2026

Thesium.finance

Plateforme de gestion de portefeuille quantitative
avec agents IA intégrés

Table des matières

1. Introduction
2. Configuration du Risque
3. Gestion des Agents IA
4. Référence API Complète
5. Schéma de Base de Données
6. Gestion des Instruments
7. Moteur d'Exécution
8. Journal d'Audit et Monitoring
9. Maintenance

1. Introduction

1.1 Rôle de l'administrateur Thesium Desk

L'administrateur Thesium Desk est responsable de la configuration, de la supervision et de la maintenance de la plateforme de gestion de portefeuille quantitative. Ses responsabilités incluent :

- La configuration et l'ajustement des paramètres de risque du portefeuille
- La supervision des agents IA de recherche et de leurs recommandations
- La gestion des instruments financiers suivis par la plateforme
- Le monitoring du journal d'audit et la conformité réglementaire
- La maintenance de la base de données et des sauvegardes
- La configuration du moteur d'exécution (paper trading ou trading réel)

1.2 Architecture technique

Thesium Desk repose sur une architecture à trois couches, conçue pour la simplicité de déploiement et la performance :

- **Backend** : API REST développée avec FastAPI (Python 3.11+), servant l'ensemble de la logique métier, des agents IA et du moteur de risque.
- **Frontend** : Interface utilisateur en Vanilla JavaScript (HTML/CSS/JS), sans framework, communiquant avec le backend via des appels API REST.
- **Base de données** : SQLite avec mode WAL activé, offrant une solution embarquée performante sans serveur de base de données externe.

1.3 Vue d'ensemble des composants

Le système se compose des modules fonctionnels suivants :

Composant	Technologie	Rôle
API REST	FastAPI / Uvicorn	Point d'entrée de toutes les opérations
Agents IA	Python (analyse quantitative)	Analyse macro, facteurs, microstructure, données alternatives
Moteur de Risque	Python	Contrôle des limites, calcul VaR, stop-loss
Moteur d'Exécution	PaperBroker / BrokerAdapter	Simulation ou exécution réelle des ordres
Base de Données	SQLite (WAL)	Persistance de toutes les données
Ingestion	Yahoo Finance (yfinance)	Récupération des données de marché OHLCV

Composant	Technologie	Rôle
Mémos IC	Générateur Markdown	Documentation des décisions d'investissement

2. Configuration du Risque

Le moteur de risque de Thesium Desk contrôle en permanence l'exposition du portefeuille. Chaque ordre passe par une vérification de risque avant exécution. Les paramètres sont configurables via l'API REST.

2.1 Paramètres par défaut

Paramètre	Valeur	Description
max_position_pct	10 %	Exposition maximale par position individuelle
max_sector_pct	25 %	Concentration maximale par secteur
max_single_name_pct	10 %	Exposition maximale par nom
max_var_pct	5 %	VaR portfolio maximum (95 %, 1 jour)
stop_loss_pct	8 %	Seuil de stop-loss par position

2.2 Modification via API

La configuration du risque se modifie via un appel PUT à l'endpoint `/api/risk-config`. Les paramètres sont transmis en tant que query parameters. Seuls les paramètres spécifiés sont mis à jour ; les autres conservent leur valeur actuelle.

Exemple : modification de deux paramètres

```
curl -X PUT "http://localhost:8000/api/risk-config?\nmax_position_pct=12&max_var_pct=4"
```

Réponse attendue (JSON)

```
{\n  "max_position_pct": 12.0,\n  "max_sector_pct": 25.0,\n  "max_single_name_pct": 10.0,\n  "max_var_pct": 4.0,\n  "stop_loss_pct": 8.0\n}
```

2.3 Consultation de la configuration

Pour consulter la configuration actuelle du risque :

```
curl http://localhost:8000/api/risk-config
```

2.4 Calcul de la Value at Risk (VaR)

Le moteur de risque utilise le calcul VaR paramétrique avec les caractéristiques suivantes :

- **Fenêtre de volatilité** : 21 jours de trading (environ 1 mois)
- **Niveau de confiance** : 95 % (z-score = 1.645)
- **Horizon** : 1 jour
- **Formule** : $VaR = \text{Valeur du portefeuille} \times \text{volatilité} \times \text{z-score}$

Note : La VaR est recalculée après chaque exécution d'ordre et lors du cycle de décision complet. Si la VaR dépasse le seuil `max_var_pct`, les nouveaux ordres seront rejetés par le contrôle de risque.

3. Gestion des Agents IA

Thesium Desk intègre quatre agents d'analyse quantitative. Chaque agent analyse les données de marché sous un angle spécifique et produit des thèses d'investissement avec un score de conviction.

3.1 MacroAgent — Analyse macroéconomique

Le MacroAgent évalue l'environnement macroéconomique en analysant les indices de référence (SPY, QQQ, DIA) pour déterminer le régime de marché.

Indicateurs utilisés

- Croisements de moyennes mobiles simples (SMA) 50 et 200 jours
- RSI (Relative Strength Index) sur 14 périodes
- Momentum sur 20 jours de trading

Stances possibles

- **risk-on** : Environnement favorable aux actifs risqués
- **risk-off** : Environnement défavorable, prudence recommandée
- **neutre** : Signaux mixtes, pas de biais directionnel clair

3.2 FactorAgent — Analyse factorielle

Le FactorAgent évalue chaque instrument selon des facteurs quantitatifs reconnus et attribue un score composite déterminant l'allocation.

Facteurs analysés

- **Momentum** : rendement sur 12 mois moins le dernier mois (momentum 12-1)
- **Qualité** : inverse de la volatilité (les actifs moins volatils sont privilégiés)
- **RSI** : Relative Strength Index comme indicateur de surachat/survente

Recommandations

- **surpondérer** (overweight) : Score composite élevé
- **sous-pondérer** (underweight) : Score composite faible
- **neutre** : Score intermédiaire

3.3 MicrostructureAgent — Analyse de microstructure

Le MicrostructureAgent se concentre sur les signaux techniques de court terme pour identifier des points d'entrée et de sortie optimaux.

Indicateurs utilisés

- Bandes de Bollinger sur 20 jours
- Ratio de volume 5 jours / 20 jours (détection d'activité inhabituelle)
- RSI (Relative Strength Index)
- Niveaux de support et résistance
- ATR 14 (Average True Range) pour la mesure de volatilité

Signaux

- **buy** : Signal d'achat identifié
- **sell** : Signal de vente identifié
- **hold** : Maintien de la position actuelle

3.4 AltDataAgent — Données alternatives

L'AltDataAgent exploite des signaux de données alternatives pour compléter l'analyse quantitative traditionnelle.

Méthodes d'analyse

- **Divergence prix-volume** : Détection de divergences entre l'évolution des prix et des volumes
- **Consistance de tendance** : Mesure de la régularité de la tendance haussière ou baissière
- **Z-score des rendements** : Identification de rendements statistiquement anormaux

Sortie

L'agent produit une classification de sentiment (positif, négatif, neutre) accompagnée d'un score de conviction.

3.5 Déclenchement des agents

Deux modes de déclenchement sont disponibles :

Mode analyse seule

Exécute les 4 agents de recherche sans déclencher d'ordres. Utile pour consulter les recommandations avant de valider le cycle complet.

```
curl -X POST http://localhost:8000/api/run-agents
```

Cycle de décision complet

Exécute la chaîne complète : agents → contrôle de risque → exécution des ordres → génération du mémo IC.

```
curl -X POST http://localhost:8000/api/orders/execute-cycle
```

3.6 Seuils de conviction pour l'exécution

Pour qu'un agent déclenche effectivement un ordre, deux conditions doivent être réunies :

Agent	Score minimum	Condition supplémentaire	Taille de l'ordre
FactorAgent	≥ 7	Tilt = "overweight"	2 % du portefeuille
MicrostructureAgent	≥ 7	Signal = "buy" ou "sell"	1 % du portefeuille

Note : Le MacroAgent et l'AltDataAgent influencent les décisions via leurs thèses et le mémo IC, mais ne déclenchent pas directement d'ordres.

4. Référence API Complète

L'ensemble des opérations de Thesium Desk est accessible via une API REST. Le serveur écoute par défaut sur `http://localhost:8000`. Toutes les réponses sont au format JSON.

Méthode	Endpoint	Description
GET	/api/health	Vérification de santé du serveur
GET	/api/dashboard	Résumé portefeuille + positions + événements
GET	/api/portfolio/history	Courbe d'équité sur 30 jours
GET	/api/theses	Liste des thèses (filtres : agent, status, ticker)
GET	/api/theses/{id}	Détail d'une thèse + ordres liés
POST	/api/theses/{id}/approve	Approuver une thèse
GET	/api/orders	Liste des ordres (filtres : status, ticker, limit)
GET	/api/fills	Liste des exécutions récentes
POST	/api/orders/execute-cycle	Cycle de décision complet
GET	/api/memos	Liste mémos IC (filtres : date, ticker, agent)
GET	/api/memos/{id}	Détail du mémo complet
GET	/api/memos/{id}/markdown	Export mémo en Markdown
GET	/api/events	Journal d'événements (filtres : event_type, agent)
POST	/api/run-ingestion	Ingestion données Yahoo Finance
POST	/api/run-agents	Exécution des agents de recherche
GET	/api/risk-config	Configuration risque actuelle
PUT	/api/risk-config	Mise à jour de la configuration risque
GET	/api/instruments	Liste des instruments suivis
GET	/api/instruments/{ticker}/prices	Historique des prix d'un instrument

4.1 Exemples d'utilisation

Vérification de santé

```
curl http://localhost:8000/api/health

# Réponse :
{"status": "ok", "timestamp": "2026-03-04T15:00:00Z"}
```

Consultation du tableau de bord

```
curl http://localhost:8000/api/dashboard
```

```
# Réponse (extrait) :  
{  
  "portfolio": {  
    "total_value": 1000000.0,  
    "cash": 850000.0,  
    "total_pnl": 2500.0,  
    "var_95": 3.2  
  },  
  "positions": [...],  
  "recent_events": [...]  
}
```

Filtrer les thèses par agent

```
curl "http://localhost:8000/api/theses?agent=FactorAgent&status=active"
```

Approuver une thèse

```
curl -X POST http://localhost:8000/api/theses/42/approve
```

Exporter un mémo en Markdown

```
curl http://localhost:8000/api/memos/7/markdown > memo_20260304.md
```

5. Schéma de Base de Données

Thesium Desk utilise une base SQLite unique (thesium.db) contenant 12 tables. Le mode WAL (Write-Ahead Logging) est activé pour optimiser les performances en accès concurrent.

5.1 instruments

Référentiel des instruments financiers suivis par la plateforme.

```
CREATE TABLE instruments (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  ticker TEXT NOT NULL UNIQUE,  
  name TEXT NOT NULL,  
  sector TEXT,  
  asset_class TEXT DEFAULT 'equity'  
);
```

5.2 prices

Historique des prix OHLCV par instrument et par date.

```
CREATE TABLE prices (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  instrument_id INTEGER REFERENCES instruments(id),  
  date TEXT NOT NULL,  
  open REAL, high REAL, low REAL, close REAL,  
  volume INTEGER  
);
```

5.3 theses

Thèses d'investissement générées par les agents IA.

```
CREATE TABLE theses (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  instrument_id INTEGER REFERENCES instruments(id),  
  agent_type TEXT NOT NULL,  
  thesis_text TEXT,  
  conviction_score REAL,  
  horizon TEXT,  
  key_drivers TEXT, -- JSON  
  proposed_action TEXT,  
  status TEXT DEFAULT 'active',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5.4 portfolio_positions

Positions actuelles du portefeuille.

```
CREATE TABLE portfolio_positions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  instrument_id INTEGER REFERENCES instruments(id),  
  quantity REAL NOT NULL,  
  avg_cost REAL NOT NULL,  
  current_price REAL,  
  unrealized_pnl REAL,  
  weight_pct REAL  
);
```

5.5 portfolio_state

État global du portefeuille (ligne unique, mise à jour après chaque opération).

```
CREATE TABLE portfolio_state (  
  id INTEGER PRIMARY KEY CHECK (id = 1),  
  cash REAL NOT NULL,  
  total_value REAL NOT NULL,  
  total_pnl REAL DEFAULT 0,  
  daily_pnl REAL DEFAULT 0,  
  var_95 REAL DEFAULT 0,  
  max_drawdown REAL DEFAULT 0  
);
```

5.6 orders

Ordres d'achat et de vente, liés aux thèses qui les ont déclenchés.

```
CREATE TABLE orders (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  instrument_id INTEGER REFERENCES instruments(id),  
  thesis_id INTEGER REFERENCES theses(id),  
  side TEXT NOT NULL, -- 'buy' ou 'sell'  
  quantity REAL NOT NULL,  
  order_type TEXT DEFAULT 'market',  
  limit_price REAL,  
  status TEXT DEFAULT 'pending',  
  risk_check_result TEXT, -- JSON  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5.7 fills

Exécutions réelles (ou simulées) des ordres.

```
CREATE TABLE fills (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  order_id INTEGER REFERENCES orders(id),  
  fill_price REAL NOT NULL,  
  fill_quantity REAL NOT NULL,  
  slippage REAL DEFAULT 0,  
  fees REAL DEFAULT 0,  
  filled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5.8 ic_memos

Mémos du Comité d'Investissement (IC), générés automatiquement à chaque cycle.

```
CREATE TABLE ic_memos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  date TEXT NOT NULL,  
  title TEXT,  
  macro_summary TEXT,  
  factor_tilts TEXT, -- JSON  
  thesis_summaries TEXT, -- JSON  
  proposed_changes TEXT, -- JSON  
  full_markdown TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5.9 event_log

Journal d'audit append-only, traçant toutes les actions du système.

```
CREATE TABLE event_log (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
event_type TEXT NOT NULL,  
entity_type TEXT,  
entity_id INTEGER,  
details TEXT, -- JSON  
agent TEXT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5.10 risk_config

Configuration des limites de risque (ligne unique).

```
CREATE TABLE risk_config (  
id INTEGER PRIMARY KEY CHECK (id = 1),  
max_position_pct REAL DEFAULT 10.0,  
max_sector_pct REAL DEFAULT 25.0,  
max_single_name_pct REAL DEFAULT 10.0,  
max_var_pct REAL DEFAULT 5.0,  
stop_loss_pct REAL DEFAULT 8.0  
);
```

5.11 portfolio_history

Historique quotidien de la valeur du portefeuille pour la courbe d'équité.

```
CREATE TABLE portfolio_history (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
date TEXT NOT NULL,  
total_value REAL NOT NULL,  
cash REAL,  
total_pnl REAL  
);
```

6. Gestion des Instruments

6.1 Instruments pré-configurés

Thesium Desk est livré avec 14 instruments pré-configurés couvrant les principales valeurs technologiques, financières, énergétiques et de santé américaines, ainsi que deux ETF indiciels :

Ticker	Secteur	Catégorie
AAPL	Technology	Equity
MSFT	Technology	Equity
GOOGL	Technology	Equity
AMZN	Consumer Cyclical	Equity
NVDA	Technology	Equity
TSLA	Consumer Cyclical	Equity
META	Technology	Equity
JPM	Financial Services	Equity
BAC	Financial Services	Equity
XOM	Energy	Equity
JNJ	Healthcare	Equity
UNH	Healthcare	Equity
SPY	Index	ETF
QQQ	Index	ETF

6.2 Consultation des instruments

```
curl http://localhost:8000/api/instruments

# Réponse (extrait) :
[
  {"id": 1, "ticker": "AAPL", "name": "Apple Inc.",
  "sector": "Technology", "asset_class": "equity"},
  ...
]
```

6.3 Ingestion des données de prix

L'ingestion des données de marché utilise Yahoo Finance via la bibliothèque `yfinance`. Elle récupère les données OHLCV (Open, High, Low, Close, Volume) pour l'ensemble des instruments suivis.

```
curl -X POST http://localhost:8000/api/run-ingestion
```

Note : L'ingestion est recommandée avant chaque cycle de décision pour garantir que les agents travaillent sur des données à jour.

6.4 Consulter l'historique des prix

```
curl http://localhost:8000/api/instruments/AAPL/prices

# Réponse (extrait) :
[
{"date": "2026-03-03", "open": 178.50, "high": 180.20,
"low": 177.80, "close": 179.90, "volume": 52340000},
...
]
```

6.5 Ajout d'un nouvel instrument

L'ajout d'instruments se fait actuellement par insertion directe en base de données. Un endpoint API dédié est prévu dans une version future.

```
import sqlite3

conn = sqlite3.connect("thesium.db")
conn.execute(
    "INSERT INTO instruments (ticker, name, sector, asset_class) "
    "VALUES (?, ?, ?, ?)",
    ("V", "Visa Inc.", "Financial Services", "equity")
)
conn.commit()
conn.close()
```

Important : Après l'ajout d'un instrument, exécutez l'ingestion (POST /api/run-ingestion) pour charger son historique de prix.

7. Moteur d'Exécution

7.1 Mode Paper Trading (défaut)

Par défaut, Thesium Desk fonctionne en mode paper trading via le composant PaperBroker. Ce mode simule l'exécution des ordres sans interaction avec un courtier réel.

Paramètres de simulation

Paramètre	Valeur	Description
Slippage	0.1 %	Glissement de prix simulé sur chaque exécution
Frais	\$0.005 / action	Commission par action simulée

7.2 Pipeline d'exécution

Chaque ordre suit un pipeline séquentiel strict avant d'être exécuté :

- **1. Contrôle de risque** : Vérification des limites (position, secteur, VaR, stop-loss). Si une limite est dépassée, l'ordre est rejeté.
- **2. Création de l'ordre** : Enregistrement dans la table orders avec le statut pending.
- **3. Exécution (fill)** : Le broker (paper ou réel) exécute l'ordre. Le prix d'exécution inclut le slippage et les frais.
- **4. Mise à jour des positions** : La table portfolio_positions est mise à jour (quantité, coût moyen, P&L non réalisé).
- **5. Rafraîchissement de l'état** : La table portfolio_state est recalculée (valeur totale, cash, VaR, drawdown).

7.3 Interface BrokerAdapter

L'architecture du moteur d'exécution repose sur l'interface abstraite BrokerAdapter, permettant de remplacer le PaperBroker par un connecteur vers un courtier réel.

```
class BrokerAdapter(ABC):
    """Interface abstraite pour les courtiers."""

    @abstractmethod
    async def execute_order(
        self, ticker: str, side: str,
        quantity: float, order_type: str,
        limit_price: float = None
    ) -> Fill:
        """Exécute un ordre et retourne un Fill."""
        pass
```

```
@abstractmethod
async def get_account_info(self) -> dict:
    """Retourne les informations du compte."""
    pass
```

Courtiers compatibles prévus :

- **Interactive Brokers** : Via l'API TWS/IB Gateway
- **Alpaca** : Via l'API REST Alpaca Markets

7.4 Passage en trading réel

Pour activer le trading réel, les étapes suivantes sont nécessaires :

- 1. Implémenter une classe héritant de `BrokerAdapter` pour le courtier choisi
- 2. Configurer la variable d'environnement `LIVE_TRADING=true`
- 3. Fournir les identifiants du courtier (clé API, secret) via les variables d'environnement
- 4. Tester en environnement de pré-production avant le passage en production

```
# Variables d'environnement pour le trading réel
export LIVE_TRADING=true
export BROKER_API_KEY="votre_cle_api"
export BROKER_API_SECRET="votre_secret"
export BROKER_ENDPOINT="https://api.courtier.com"
```

■ **Avertissement** : Le passage en trading réel implique des risques financiers. Assurez-vous d'avoir testé exhaustivement la configuration en mode paper trading et d'avoir validé les limites de risque avant de procéder.

8. Journal d'Audit et Monitoring

8.1 Table event_log

La table event_log constitue le journal d'audit central de Thesium Desk. Elle est de type append-only : aucun enregistrement n'est modifié ni supprimé, garantissant la traçabilité complète de toutes les actions du système.

8.2 Types d'événements

Type d'événement	Description	Agent concerné
agent_thesis_generated	Thèse d'investissement générée par un agent	Tous
risk_check	Résultat du contrôle de risque sur un ordre	—
order_created	Nouvel ordre créé dans le système	FactorAgent, MicrostructureAgent
order_filled	Ordre exécuté (fill enregistré)	—
thesis_approved	Thèse approuvée par l'administrateur	—
agent_cycle_complete	Cycle complet de décision terminé	Tous

8.3 Consultation des événements

L'endpoint GET /api/events permet de consulter le journal avec des filtres optionnels :

Tous les événements récents

```
curl http://localhost:8000/api/events
```

Filtrer par type d'événement

```
curl "http://localhost:8000/api/events?event_type=order_filled"
```

Filtrer par agent

```
curl "http://localhost:8000/api/events?agent=FactorAgent"
```

Exemple de réponse

```
[  
  {  
    "id": 156,
```

```
"event_type": "order_filled",
"entity_type": "order",
"entity_id": 42,
"details": {
  "ticker": "AAPL",
  "side": "buy",
  "quantity": 55,
  "fill_price": 179.72
},
"agent": "FactorAgent",
"created_at": "2026-03-04T14:30:00Z"
}
]
```

8.4 Mémos IC comme preuve de conformité

Les mémos du Comité d'Investissement (IC) sont générés automatiquement à chaque cycle de décision complet. Ils documentent :

- Le résumé macro-économique du MacroAgent
- Les tilts factoriels recommandés par le FactorAgent
- Les résumés des thèses actives par instrument
- Les modifications de positions proposées et exécutées

Ces mémos constituent une piste d'audit complète pour la conformité réglementaire. Ils sont exportables en Markdown via l'endpoint GET `/api/memos/{id}/markdown`.

9. Maintenance

9.1 Sauvegarde de la base de données

La base SQLite peut être sauvegardée par simple copie du fichier. Il est recommandé d'effectuer une sauvegarde quotidienne :

```
cp thesium.db thesium_backup_$(date +%Y%m%d).db
```

Pour automatiser la sauvegarde, ajoutez une tâche cron :

```
# Sauvegarde quotidienne à 23h00
0 23 * * * cp /chemin/vers/thesium.db \
/chemin/vers/backups/thesium_backup_$(date +%Y%m%d).db
```

Note : Le mode WAL de SQLite permet de copier la base même pendant que le serveur est en cours d'exécution. Assurez-vous de copier également les fichiers thesium.db-wal et thesium.db-shm si présents.

9.2 Mode WAL SQLite

Le mode WAL (Write-Ahead Logging) est activé par défaut pour Thesium Desk. Ce mode offre les avantages suivants :

- Lectures et écritures simultanées sans blocage
- Meilleures performances en environnement concurrent
- Récupération automatique en cas de crash

Pour vérifier que le mode WAL est actif :

```
sqlite3 thesium.db "PRAGMA journal_mode;"
# Réponse attendue : wal
```

9.3 Réinitialisation des données

Pour remettre la plateforme dans son état initial avec les données de démonstration :

```
python seed_data.py

# Ce script :
# 1. Supprime et recrée toutes les tables
# 2. Insère les 14 instruments par défaut
# 3. Initialise portfolio_state (cash: $1,000,000)
# 4. Configure les paramètres de risque par défaut
# 5. Génère un historique de prix de démonstration
```

■ **Attention :** Cette opération est irréversible. Effectuez une sauvegarde avant de réinitialiser.

9.4 Configuration du serveur

Thesium Desk utilise Uvicorn comme serveur ASGI. Configuration recommandée :

```
# Démarrage standard
uvicorn main:app --host 0.0.0.0 --port 8000 --log-level info

# Démarrage en production (avec workers)
uvicorn main:app --host 0.0.0.0 --port 8000 \
--workers 4 --log-level warning

# Démarrage en mode développement (rechargement automatique)
uvicorn main:app --host 0.0.0.0 --port 8000 \
--reload --log-level debug
```

9.5 Logs et diagnostic

Les logs du serveur sont configurés via le paramètre `log_level` d'Uvicorn. Niveaux disponibles :

Niveau	Usage recommandé
debug	Développement et diagnostic détaillé
info	Production standard — suivi des requêtes et événements
warning	Production — uniquement les alertes et erreurs
error	Minimum — uniquement les erreurs critiques

Pour rediriger les logs vers un fichier :

```
uvicorn main:app --host 0.0.0.0 --port 8000 \
--log-level info 2>&1 | tee -a thesium_server.log
```

Fin du Guide Administrateur — Thesium Desk v1.0

Document généré le 4 mars 2026 — Thesium.finance